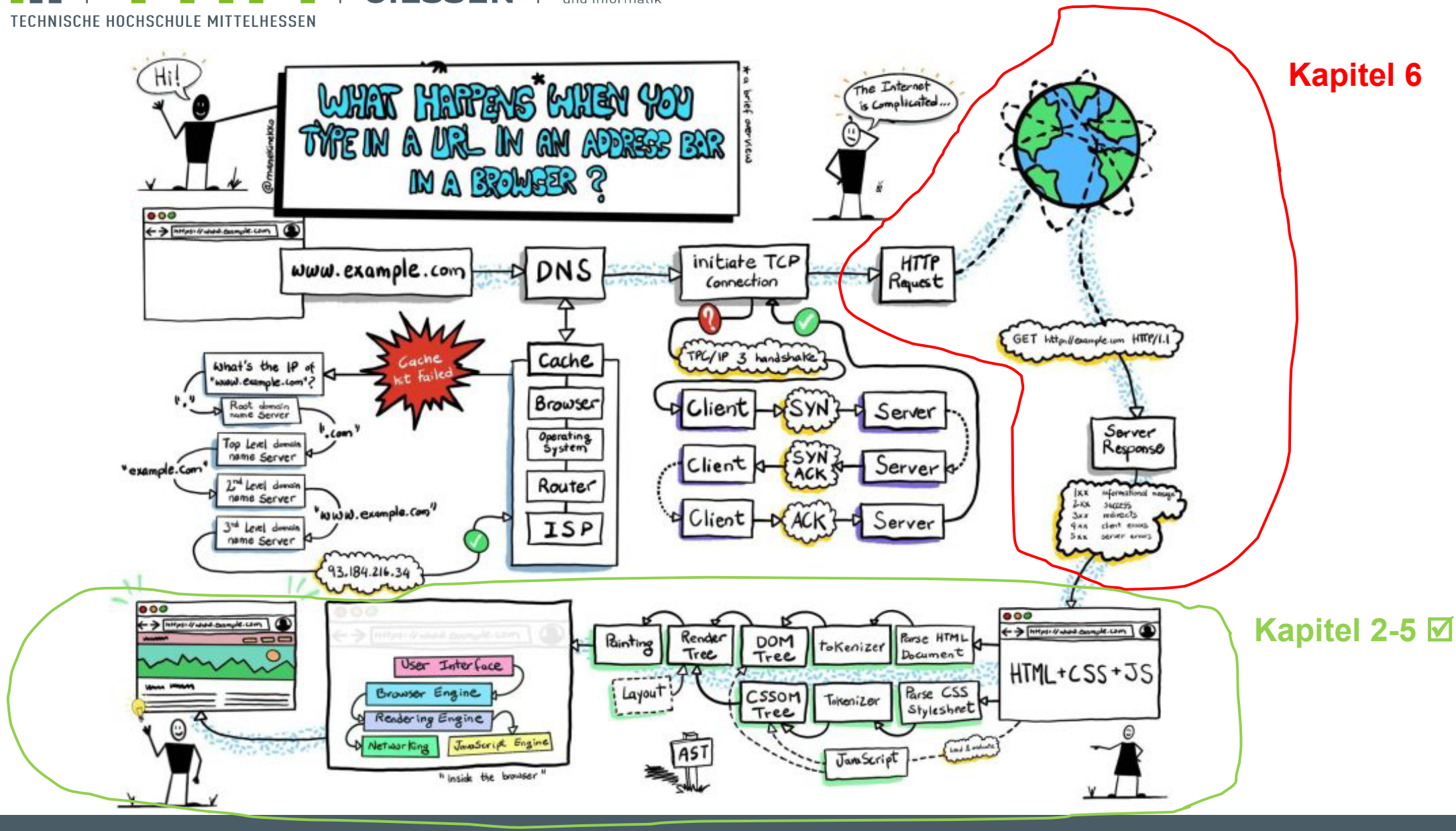
1.	Organisatorisches	4
2.	Einleitung	21
3.	HTML	46
4.	CSS	70
5.	Frameworks	89
6.	**HTTP**	**106**
7.	Internet	126
8.	TypeScript	178
9.	HTML+CSS+Typescript	224
10.	Web Services	235
11.	REST-Services	258



Kapitel 6

Kapitel 2-5 ☑

HTTP

HTTP

Hypertext

Nichtlineare, netzartige Texte.

Transfer

„Übertragung“
HTTP definiert, wie Daten im Web übertragen werden.

Protocol

„Protokoll“
„eine Vereinbarung, nach der die Datenübertragung zwischen zwei oder mehreren Parteien abläuft“[1].

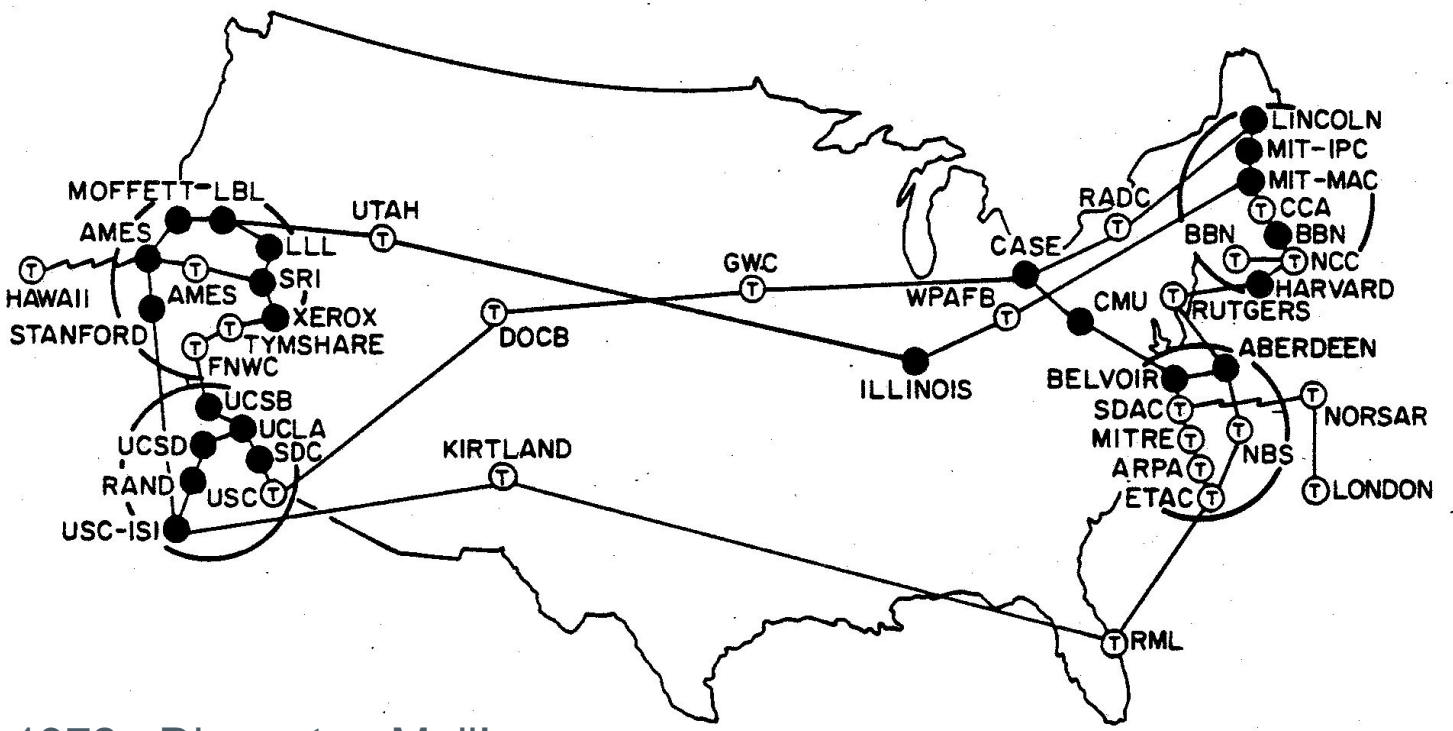
[1] https://de.wikipedia.org/wiki/Kommunikationsprotokoll

Die Anfänge …

Das ARPAnet 1969:



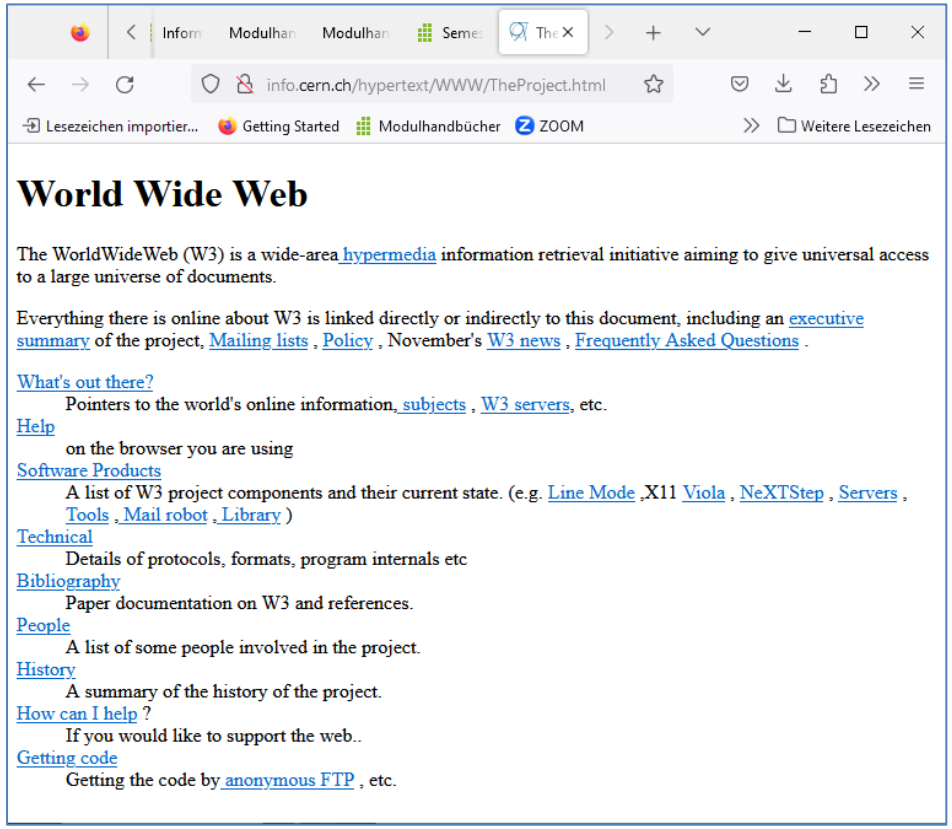
Sechs Jahre später …



1972 : Die erste eMail!
1974 : Vince Cerf und Bob Kahn erfinden TCP!

Tim-Berners Lee





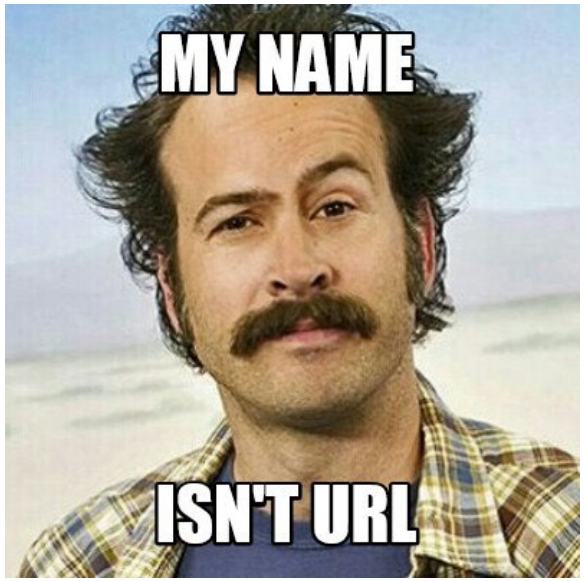
Tim Berners Lee ist ein britischer Physiker und Informatiker.

- Er entwickelte am CERN zu Beginn der 90‘er Jahre
  - HTML
  - HTTP
  - URL
  - Den ersten Web Browser
  - Den ersten Web Server
- Erstellte die erste Web-Site info.cern.ch (siehe rechts)
- Gründete 1994 das W3C
- Professor am MIT und in Oxford

Was möchten wir Anfragen?

Uniform **R**esource **L**ocator (URL):

- Host und Ressource zusammen werden als URL („Uniform Resource Locator“) bezeichnet.
- Die URL bestimmt eindeutig, was der/die Benutzer*in sehen möchte.



https://www.memecreator.org/meme/my-name-isnt-url

Protokoll

Wie wollen wir mit dem Server kommunizieren?

Servername

Mit welchem Server möchten wir sprechen?

(Wird durch DNS zu einer IP-Adresse)

Pfad

Welche spezifische Ressource auf dem Server wollen wir abrufen?

HTTP im Detail

Benutzer fragt www.thm.de/site/ an:

1a. HTTP **Client** initiiert TCP Verbindung zum HTTP Server Prozess auf dem Rechner www.thm.de auf Port 80

1b. HTTP **Server** auf www.thm.de wartet auf TCP Verbindung an Port 80; “akzeptiert” Verbindung und informiert den Client darüber

2. HTTP Client sendet eine HTTP Anfragenachricht (*request message)* (inkl. URL) an einen TCP Socket. Die Nachricht besagt, dass der Client das Objekt /site/ anfragt

3. HTTP Server empfängt die Anfrage, baut eine Antwortnachricht (*response message),* die das angefragte Objekt enthält und sendet die Nachricht an seinen Socket

4. HTTP Server schließt die TCP Verbindung

5. HTTP Client empfängt die Antwortnachricht, parst die HTML-Datei identifiziert noch anzufragende Objekte (z.B. Bilder)

6. Schritte 1-5 werden für alle Objekte wiederholt (!)

Zeit

Persistente- und Nicht-persistente Verbindungen

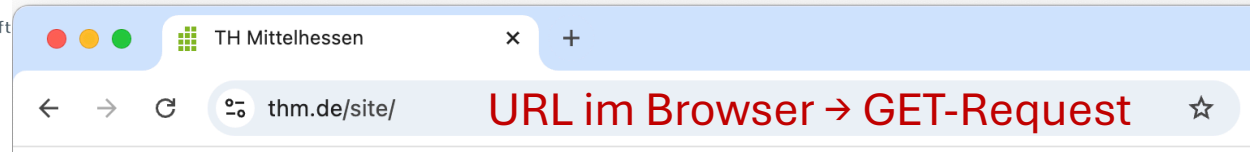
Frage: Was ist der größte Nachteil des eben gezeigten Protokolls?

Es werden n Verbindungen aufgebaut und geschlossen – eine würde reichen!

Seit HTTP 1.1 kann der Client den Server bitten, die Verbindung nicht zu schließen, so dass er sie für mehrere Anfragen nutzen kann. Man spricht dann von **persistenten Verbindungen.**

Dies wird durch den **Keep-Alive** Header spezifiziert.

NB: In http/2 und http/3 ist der Parameter nicht erlaubt.



HTTP-Methoden

GET

Fordert Daten vom Server an.

Beispiel:

GET /users/me/media
(Liste meiner eigenen Fotos auf Insta)

POST

Sendet Daten an den Server.

Beispiel:

POST /users/me/media
(Postet ein Foto oder Video)

PUT

Aktualisiert bestehende Daten auf dem Server *vollständig*.

Beispiel:

PUT /media/{media-id}
(Ändert z.B. die Beschreibung eines Fotos)

PATCH

Aktualisiert bestehenden Daten auf dem Server nur *teilweise*.

DELETE

Entfernt bestehende Daten.

Beispiel:

DELETE /media/{media-id}

HEAD

Ruft die Header-Informationen einer Ressource ohne Inhalt ab.

Beispiel einer Request Message

```
> curl --trace-ascii curl.trace http://www.google.com/
> cat curl.trace
== Info:   Trying 216.58.208.36...
== Info: TCP_NODELAY set
== Info: Connected to www.google.com (216.58.208.36) port 80 (#0)
=> Send header, 78 bytes (0x4e)
0000: GET / HTTP/1.1
0010: Host: www.google.com
0026: User-Agent: curl/7.63.0
003f: Accept: */*
004c:
```

Request Zeile

Header Zeilen

➔ *http://curl.se*

Beispiel einer Response-Nachricht

```
HTTP/1.1 200 OK
Date: Tue, 16 Jul 2019 05:21:17 GMT
Expires: -1
Cache-Control: private, max-age=0
Content-Type: text/html; charset=ISO-8859-1
P3P: CP="This is not a P3P policy! See g.co/p3phelp for more info."
< Server: gws
X-XSS-Protection: 0
X-Frame-Options: SAMEORIGIN
Set-Cookie: …
Set-Cookie: …
Accept-Ranges: none
Vary: Accept-Encoding
Transfer-Encoding: chunked
0000: 036f
0006: <!doctype html><html itemscope="" itemtype="http://schema.org/We
0046: bPage" lang="de"><head><meta content="text/html; charset=UTF-8„
…
```

Status Zeile

Header Zeilen

Daten

HTTP Statuscode

Ein HTTP-Statuscode ist eine Antwort eines Servers auf eine HTTP-Anfrage, die den Status der Anfrage angibt.

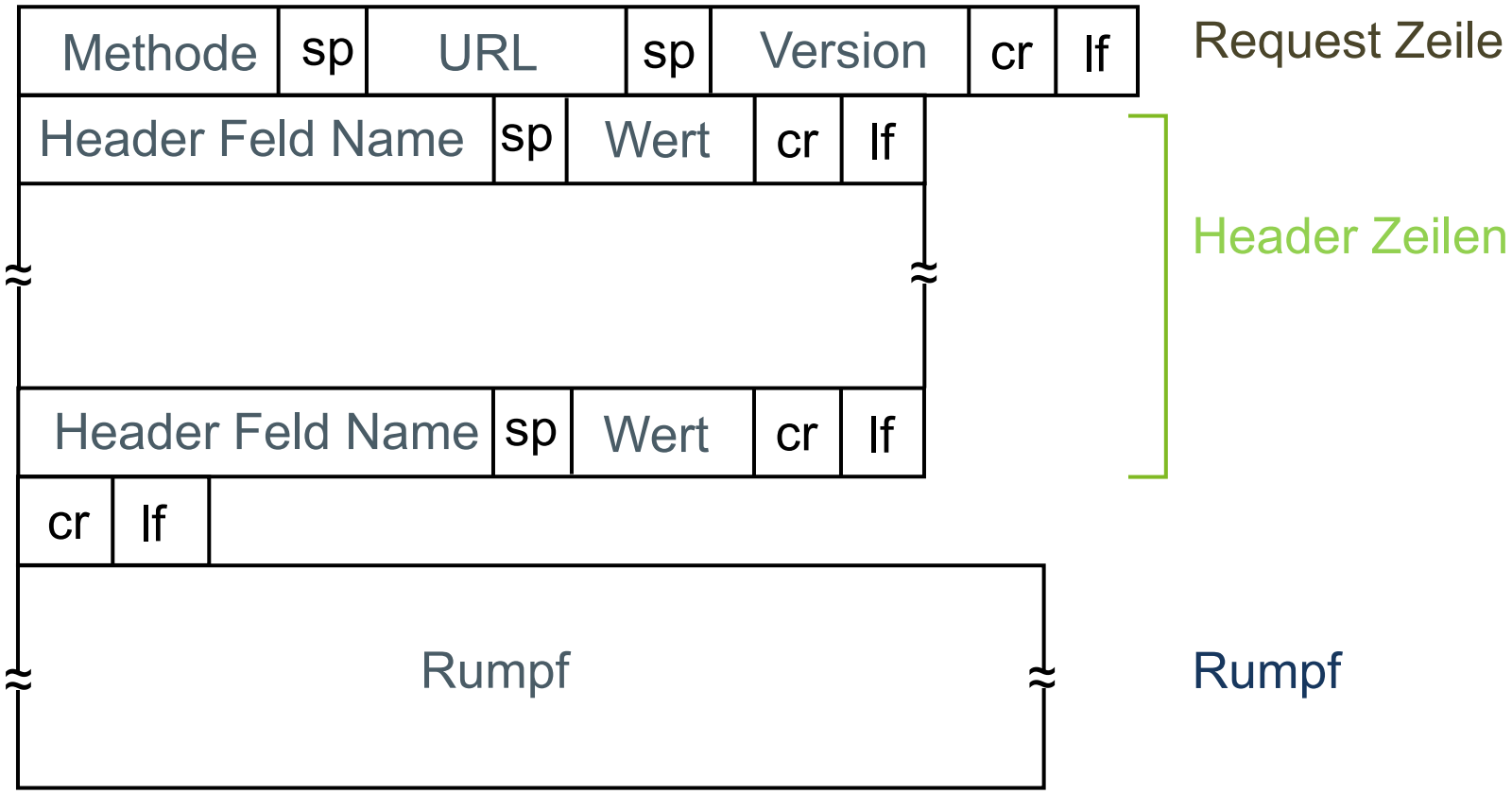
Statuscode-Gruppe	Beschreibung
1xx	Information
2xx	Erfolgreich
3xx	Umleitung (Redirects)
4xx	Client-Fehler
5xx	Server-Fehler

Häufige Statuscodes:

`200 OK`	Die Anfrage war erfolgreich.
`301 Moved Permanently`	Die angeforderte Ressource wurde dauerhaft verschoben.
`302 Found`	Die angeforderte Ressource wurde vorübergehend verschoben.
`400 Bad Request`	Die Anfrage war fehlerhaft oder konnte nicht verstanden werden.
`401 Unauthorized`	Authentifizierung ist erforderlich und fehlgeschlagen oder noch nicht bereitgestellt.
`403 Forbidden`	Der Server verweigert die Aktion aus Berechtigungsgründen.
`404 Not Found`	Die angeforderte Ressource wurde nicht gefunden.
`500 Internal Server Error`	Ein allgemeiner Fehler trat auf dem Server auf.
`503 Service Unavailable`	Der Dienst ist vorübergehend nicht verfügbar.

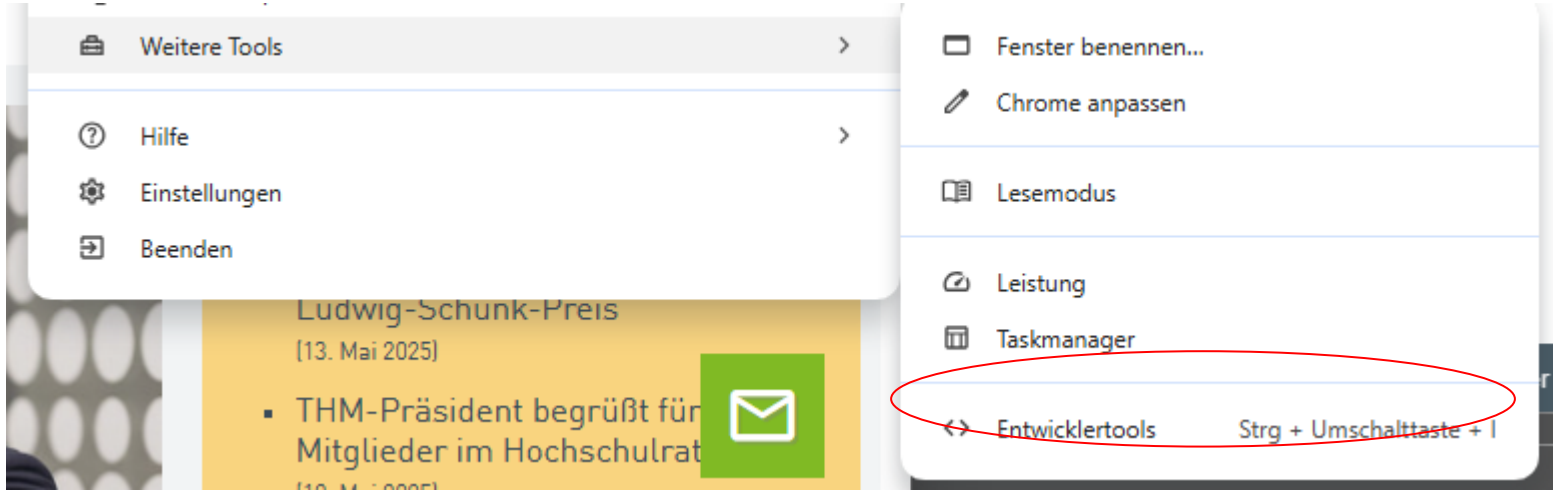
Mehr Statuscodes auf: https://developer.mozilla.org/en-US/docs/Web/HTTP/Status

Genereller Aufbau einer Request-Nachricht



Browser-Konsole

In allen Browsern können Sie Informationen über die HTTP-Verbindung, das DOM, Fehlermeldungen, … erhalten.



Tastatur-Shortcuts:
Windows: Google, Edge, Firefox: Strg + Shift + i
MacOS: Cmd + Option + I

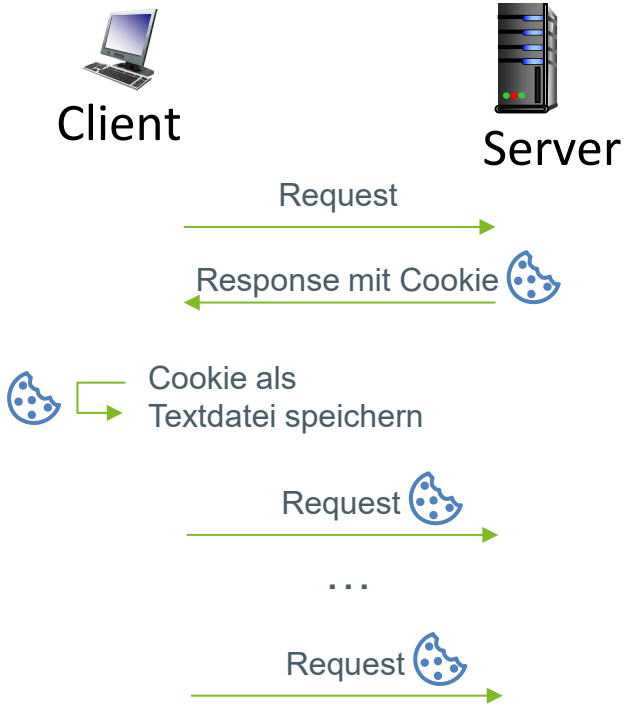
Sessions

- HTTP ist **zustandslos** ...
  - Der Server „weiß“ bei einem Request nicht, ob der vorige Request vom gleichen Client kam.
- Viele Anwendungen benötigen aber Zustandsinformationen, um z.B. ...
  - einen Nutzer eindeutig identifizieren zu können
  - eine komplexe Sitzung (**Session**) steuern zu können
- Dazu kennt man im wesentlichen drei Methoden
  - **Cookies**: Session-ID ist Teil des HTTP-Headers
  - Hidden form fields
  - URL



Cookies

- Cookies können auf zwei Arten generiert werden:
  - Sie können als Teil des HTTP-Headers **vom Server** zum Client übertragen werden.
  - Sie können **im Client** lokal generiert werden.
- Cookies werden als Textdatei auf dem System des Browsers abgelegt. Server können Cookies verwenden, um Informationen auf dem Client zu speichern.
- Der Browser überträgt diese Informationen dann bei **jedem** weiteren HTTP-Request, sofern der Browser dies zulässt.
- Der Server kann nur auf die von ihm selbst generierten Cookies zugreifen. Der Browser überträgt Cookies nur an die Server, welche den Cookie erzeugt haben.



Cookie-Attribute

- Syntax im HTTP-Response Header_
  - **`Set-Cookie: <Name>=<Wert> {; <Attribut>}`**
- mit folgenden möglichen Attributen:
  - **`EXPIRES=<date>`**
    Verfallsdatum im Format Wdy, DD-Mon-YY HH:MM:SS GMT, ohne Expire oder mit Expire <=0 ist das Cookie nur während einer Browsersitzung als "Session-Cookie" gültig.
  - **`DOMAIN=<domainname>`**
    Die Subdomäne oder Domäne als Gültigkeitsbereich des Cookies (default: Domäne, falls dieses Attribut nicht angegeben ist)
  - **`PATH=<pathname>`**
    Pfadpräfix, der den Gültigkeitsbereich auf bestimmte Pfade zu beschränken
  - **`SECURE`**
    Gibt an, dass das Cookie nur über eine sichere Verbindung zurückgeschickt wird.

Gefahren durch Cookies

Risiken

- Third party cookies: Inhalte anderer Websites (z.B. 1x1 Pixel Bilder), die der Browser dann mit dem Aufruf ebenfalls einbindet und dabei deren Cookies annimmt
- Cross-site scripting (XSS): Zugriff auf Cookies durch eingeschleusten JavaScript-code
- Tracking/Profiling: Gezieltes Nachverfolgen der Aktionen eines Benutzers.
- Cookie Banner Fatigue

Maßnahmen

Browser können:

- Cookies vollständig unterbinden
- Persistente Cookies unterbinden
- Cookies von Drittanbietern (nicht der aufgerufenen Seite) unterbinden
- Cookies nach Rückfrage an den Benutzer unterbinden
- Cookies automatisch löschen
- Private-Browsing Mode unterstützen
- XSS blockieren

Beispiel

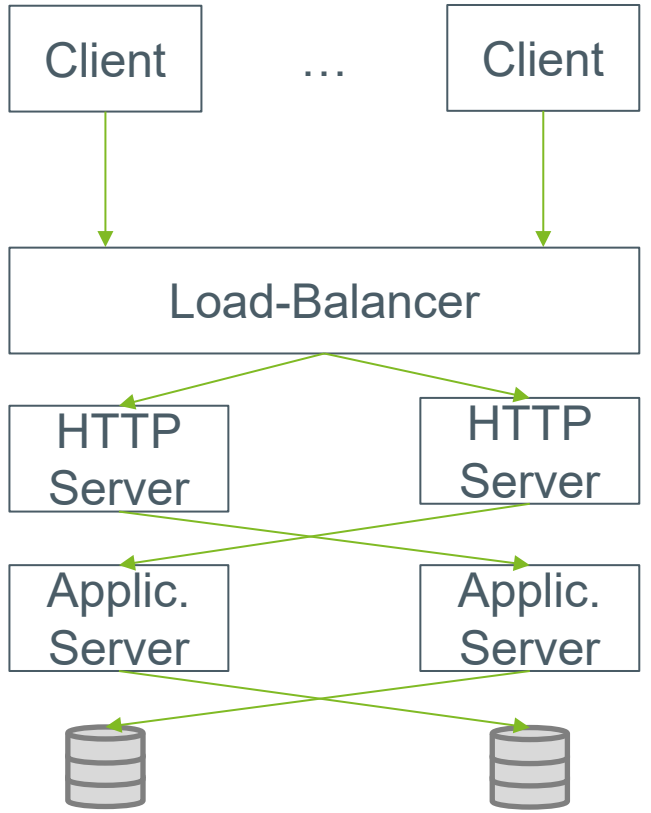
- Rufen Sie die URL https://api.predic8.de/ auf. Ist das HTML?

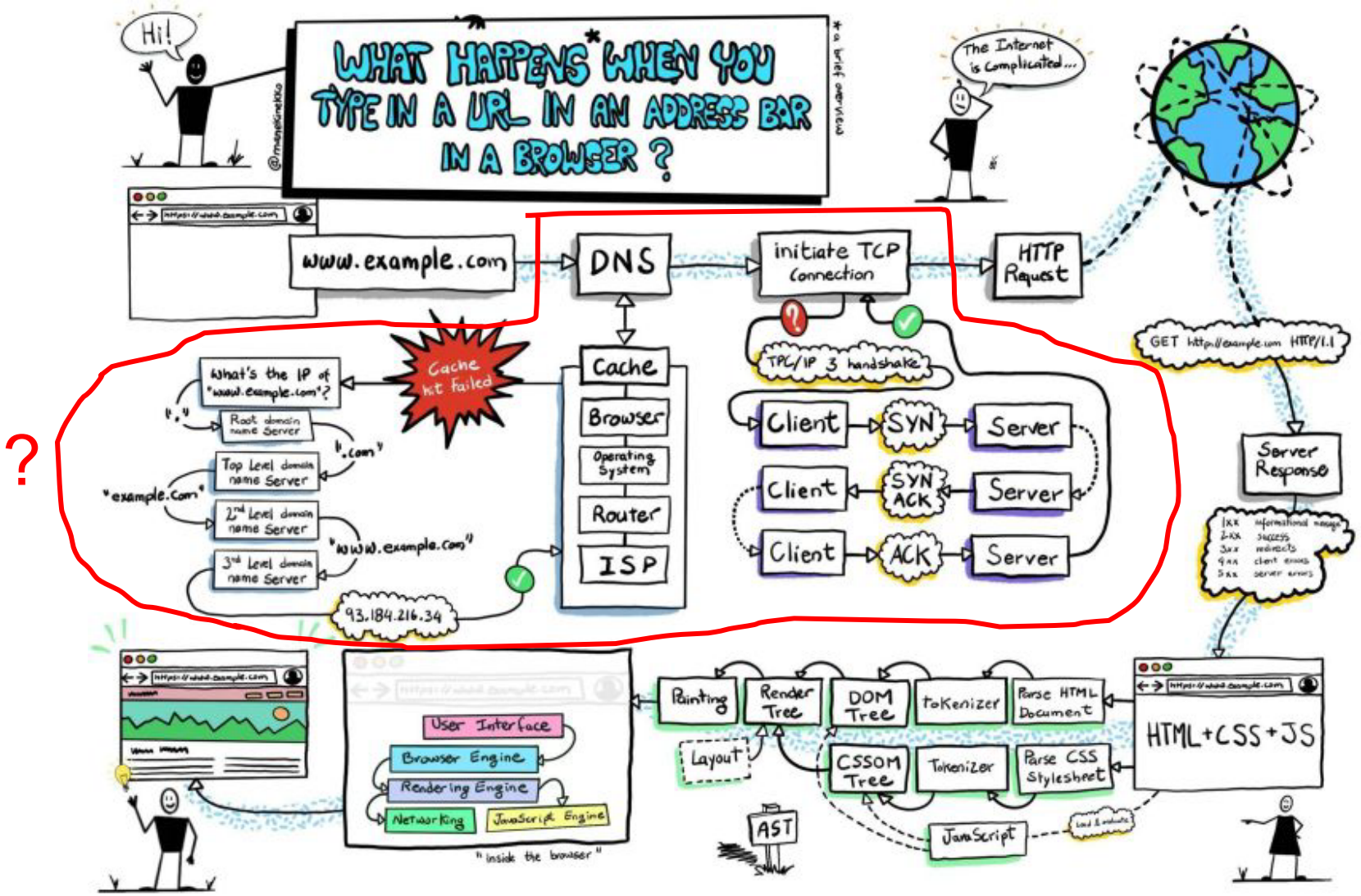
- Rufen Sie das Produkt mit der Nummer 16 ab. Wie heißt der Lieferant (‚vendor‘)?

Web Server

- Web-Server / HTTP-Server sind Programme, die das HTTP-Protokoll verstehen
- Sie „lauschen“ auf einem oder mehreren Ports (z.B. 80)
- Sie können auf das Dateisystem, Datenbanken oder weitere Programme auf dem Server zugreifen.
- Beispiele: Apache HTTP-Server, Microsoft IIS, Cloudflare.

- Auf größeren Sites sind HTTP-Server, Application-Server und Datenbank-Server meist getrennt.
- Warum?




Hi!
WHAT HAPPENS* WHEN YOU TYPE IN A URL IN AN ADDRESS BAR IN A BROWSER ?
* a brief overview
The Internet is complicated...
www.example.com
DNS
initiate TCP connection
HTTP Request
GET http://example.com HTTP/1.1
Server Response
Cache
Browser
Operating System
Router
ISP
Cache hit failed
What's the IP of "www.example.com"?
Root domain name Server
Top Level domain name Server
2nd Level domain name Server
3rd Level domain name Server
".com"
"example.com"
"www.example.com"
93.184.216.34
TPC/IP 3 handshake
Client
SYN
Server
Client
SYN ACK
Server
Client
ACK
Server
?
User Interface
Browser Engine
Rendering Engine
Networking
JavaScript Engine
"inside the browser"
Painting
Render Tree
DOM Tree
tokenizer
Parse HTML Document
Layout
CSSOM Tree
Tokenizer
Parse CSS Stylesheet
AST
JavaScript
HTML+CSS+JS